

Deep Dive into DAX User-Defined Functions

Michiel Rozema

About me

Microsoft MVP since 2019 – Power BI, Fabric
8 years Data Insight Lead at Microsoft, NL
Power BI expert, consultant, trainer, author

Founder & CEO of Quanto, Collective Analytics

Founder of Dutch Power BI User Group

Organizer of the Power BI Summer School

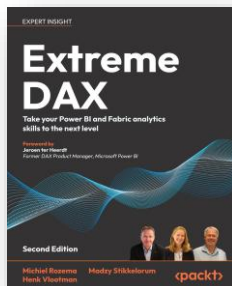
(Co-) author of several Power Pivot & Power BI books,
and video courses



Michiel.rozema@quanto.eu



www.linkedin.com/in/MichielRozema



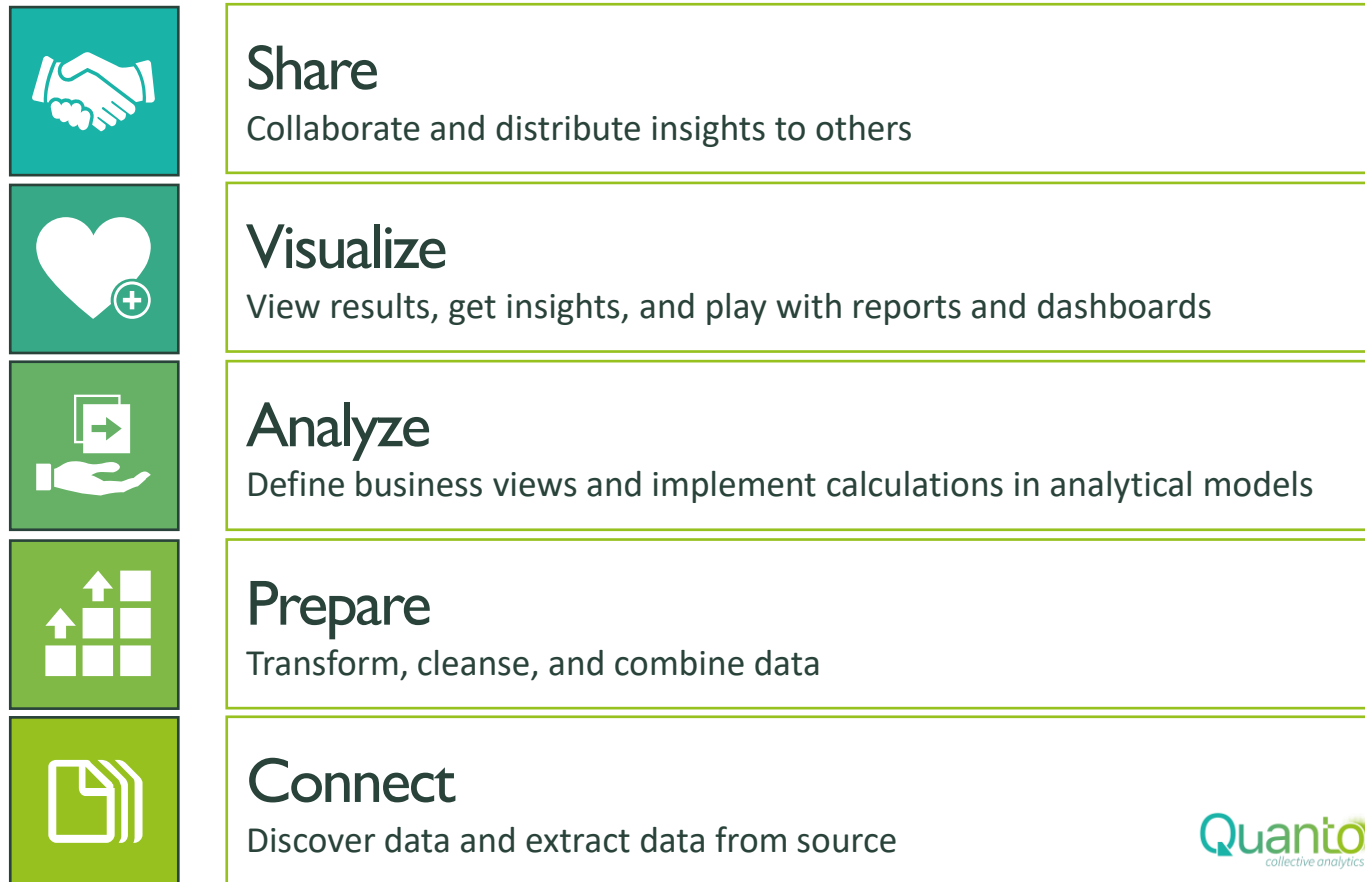
Who are you?

- What line of work are you in?
- What is your experience with Power BI?
- What is your experience with DAX?
- What do you hope to get from this workshop?

What we're going to do today

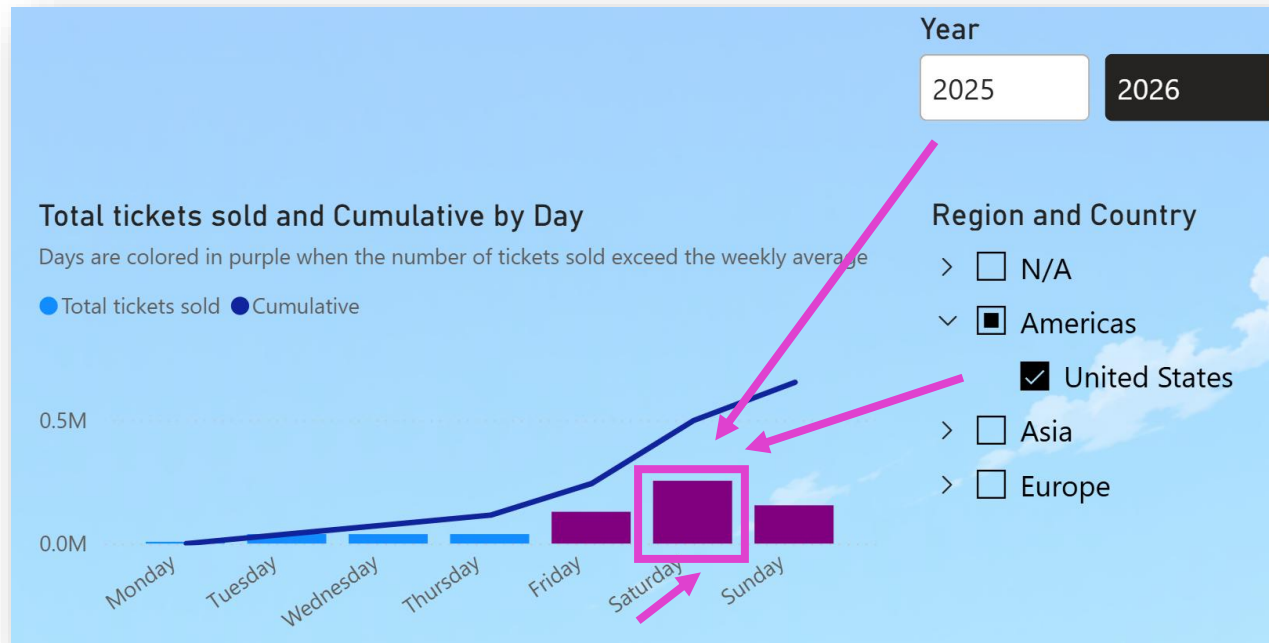
- Recap of important DAX concepts and features
- Introduction to DAX UDFs
- Parameter types and parameter passing mode
- Helper functions
- Example use cases of UDFs
- Your use case?

The Five-Layer model for Business Intelligence



DAX Evaluation Context

DAX evaluation context



Evaluation context is the most important concept to understand when working with DAX

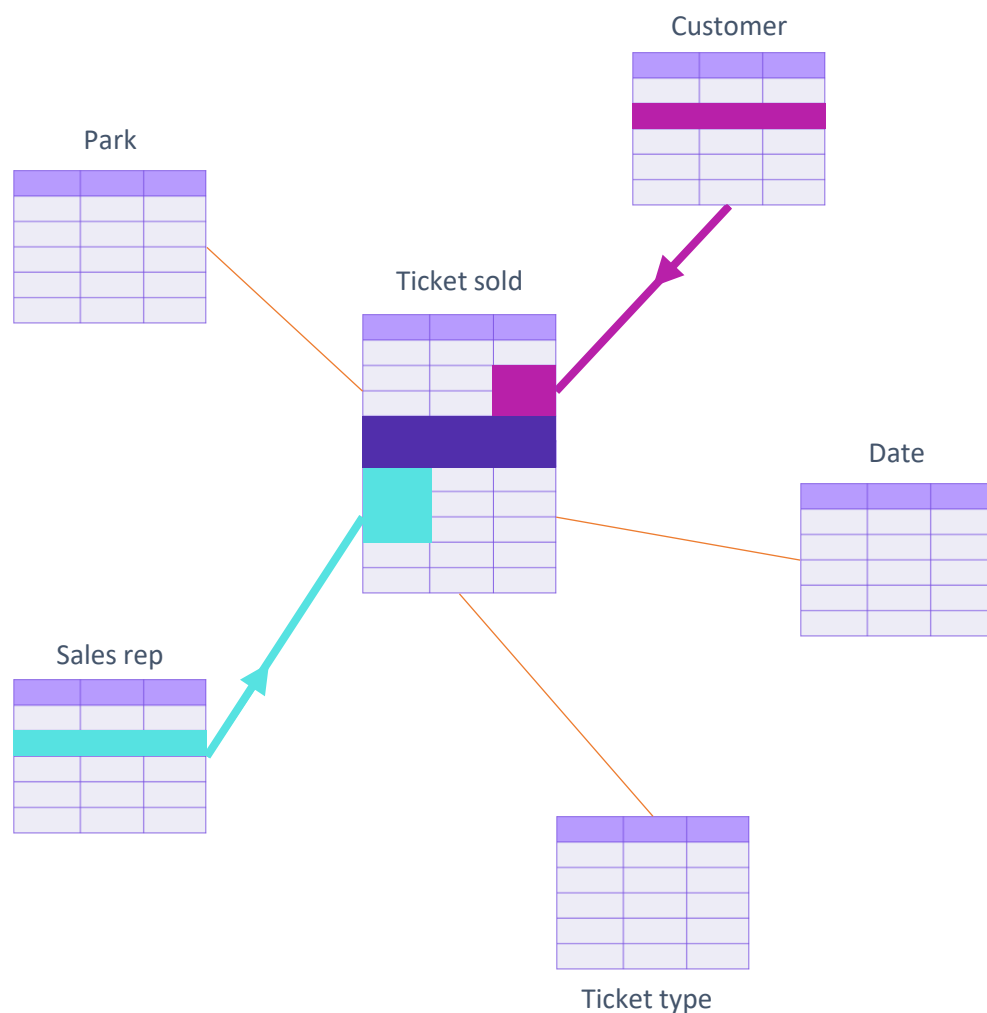
When using measures in a report, context is formed by *filters* and *selections*

Most report elements create **filters**

- Think of these as instructions to select one or more values in a specific column in the model

Filters from the report correspond to **selection** of rows in model tables

DAX evaluation context: relationships

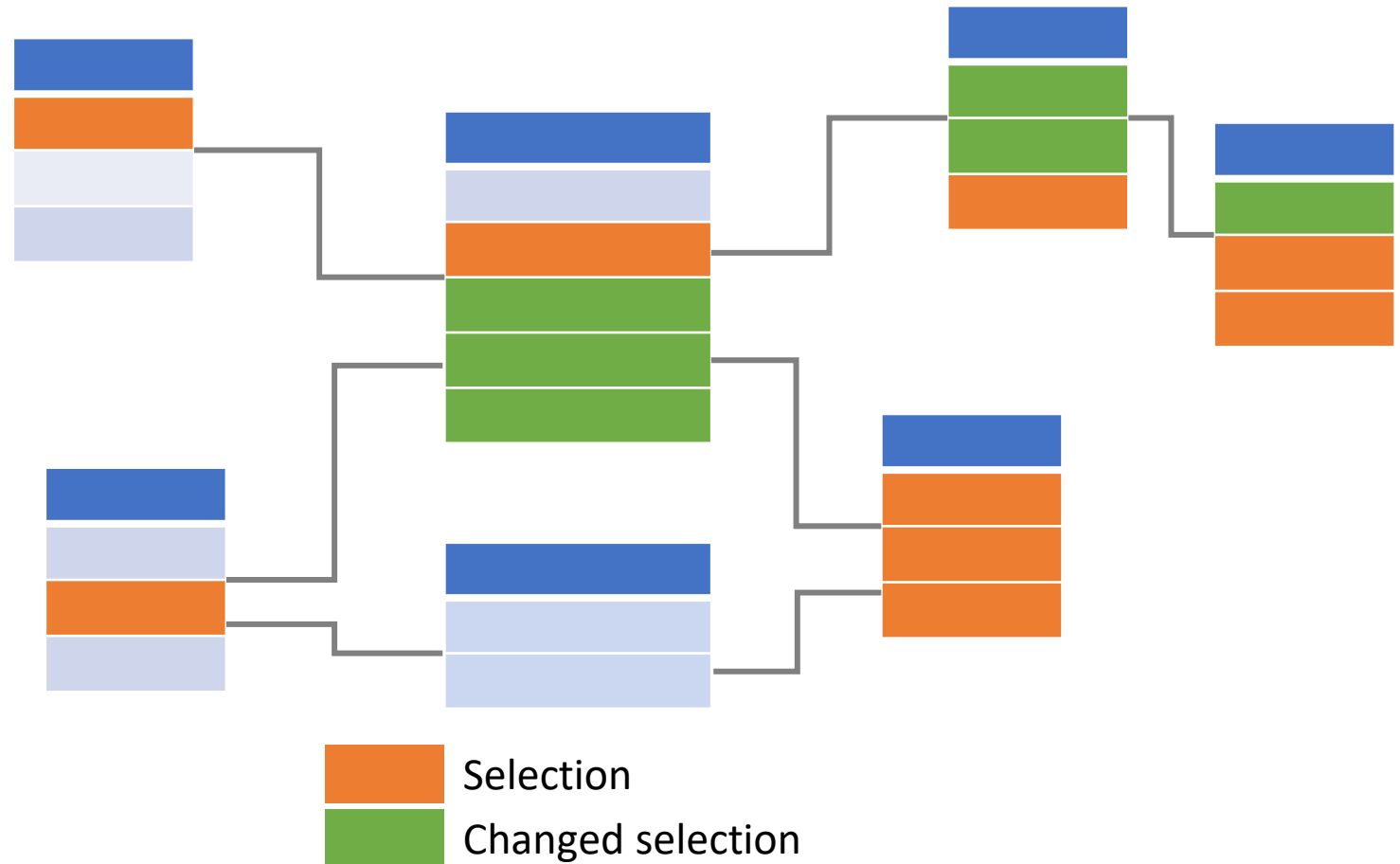


The purpose of relationships is to do **filter propagation**

- A relationship propagates the filter table's row selection in the direction of the arrow
- In the fact table, only rows are now selected that are linked to selected rows in the filter table
- All filters together result in a selected set of rows in the fact table
- The DAX measure's calculation is evaluated on that set of rows

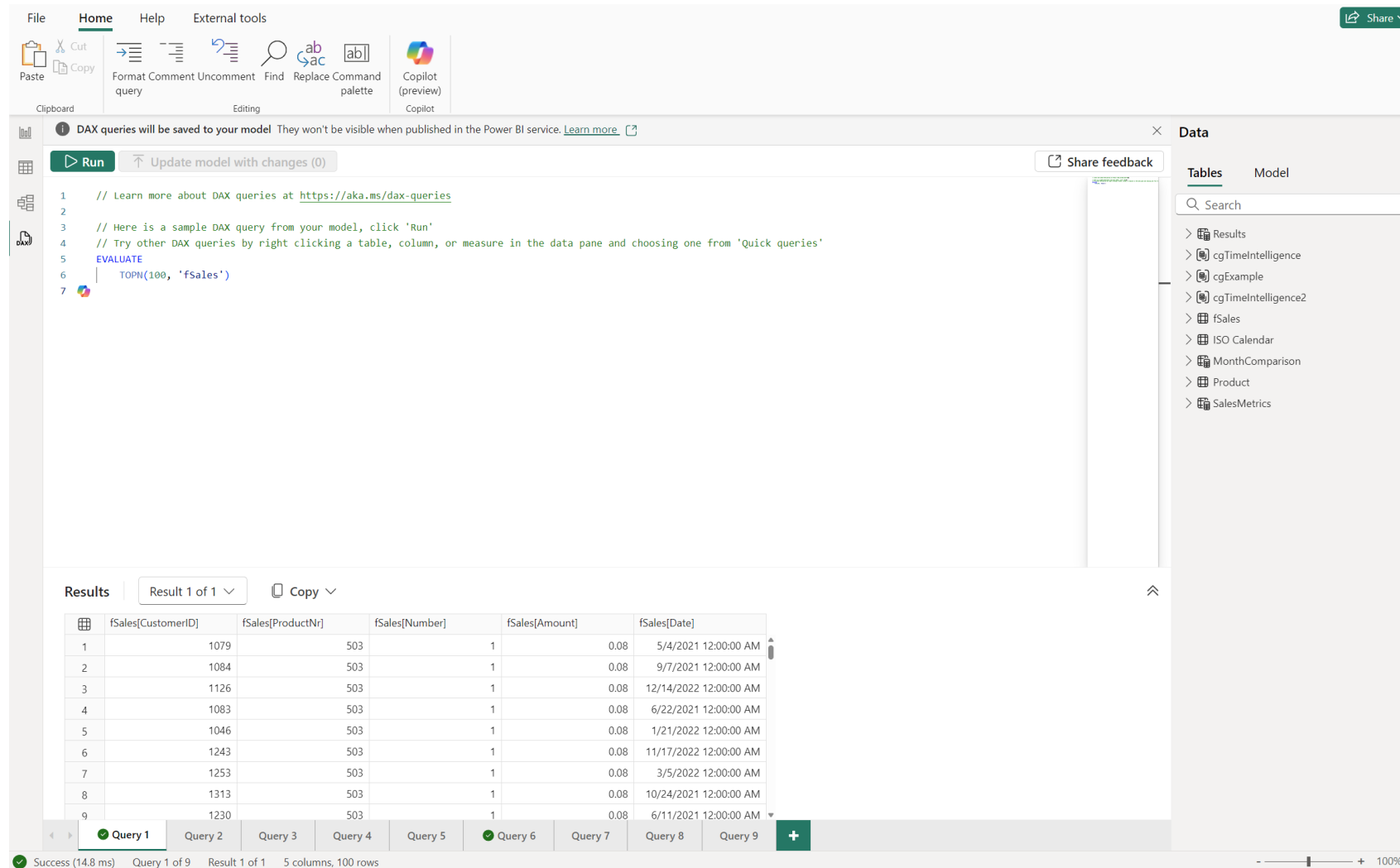
Filter context

- Context can be ***transformed with DAX***
- The DAX function to use is CALCULATE
- Or: CALCULATETABLE



DAX Query View

DAX Query View



DAX queries will be saved to your model They won't be visible when published in the Power BI service. [Learn more.](#)

Run **Update model with changes (0)** **Share feedback**

```

1 // Learn more about DAX queries at https://aka.ms/dax-queries
2
3 // Here is a sample DAX query from your model, click 'Run'
4 // Try other DAX queries by right clicking a table, column, or measure in the data pane and choosing one from 'Quick queries'
5 EVALUATE
6 | TOPN(100, 'fSales')
7

```

Results **Result 1 of 1** **Copy**

	fSales[CustomerID]	fSales[ProductNr]	fSales[Number]	fSales[Amount]	fSales[Date]
1	1079	503	1	0.08	5/4/2021 12:00:00 AM
2	1084	503	1	0.08	9/7/2021 12:00:00 AM
3	1126	503	1	0.08	12/14/2022 12:00:00 AM
4	1083	503	1	0.08	6/22/2021 12:00:00 AM
5	1046	503	1	0.08	1/21/2022 12:00:00 AM
6	1243	503	1	0.08	11/17/2022 12:00:00 AM
7	1253	503	1	0.08	3/5/2022 12:00:00 AM
8	1313	503	1	0.08	10/24/2021 12:00:00 AM
9	1230	503	1	0.08	6/11/2021 12:00:00 AM

Success (14.8 ms) Query 1 of 9 Result 1 of 1 5 columns, 100 rows

A visual's DAX query

DEFINE

```
VAR __DS0FilterTable =
    TREATAS({"Furniture"}, 'Product'[Category])
```

```
VAR __DS0Core =
    SUMMARIZECOLUMNS(
        'Product'[Product],
        __DS0FilterTable,
        "Sales", 'Results'[Sales]
    )
```

```
VAR __DS0PrimaryWindowed =
    TOPN(1001, __DS0Core, [Sales], 0, 'Product'[Product], 1)
```

EVALUATE

```
__DS0PrimaryWindowed
```

ORDER BY

```
[Sales] DESC, 'Product'[Product]
```

DEFINE: core results table

DEFINE

```
...  
VAR __DS0Core =  
    SUMMARIZECOLUMNS(  
        'Product'[Product],  
        __DS0FilterTable,  
        "Sales", 'Results'[Sales]  
    )
```

DEFINE: windowed results

DEFINE

```
...  
VAR __DS0PrimaryWindowed =  
    TOPN(1001, __DS0Core, [Sales], 0, 'Product'[Product], 1)
```

EVALUATE and ORDER BY

...

EVALUATE

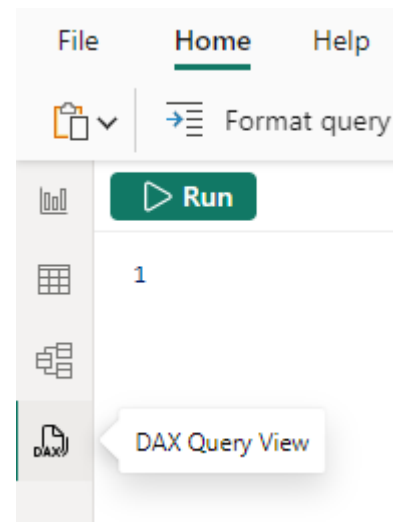
__DS0PrimaryWindowed

ORDER BY

[Sales] DESC, 'Product' [Product]

DAX Query View

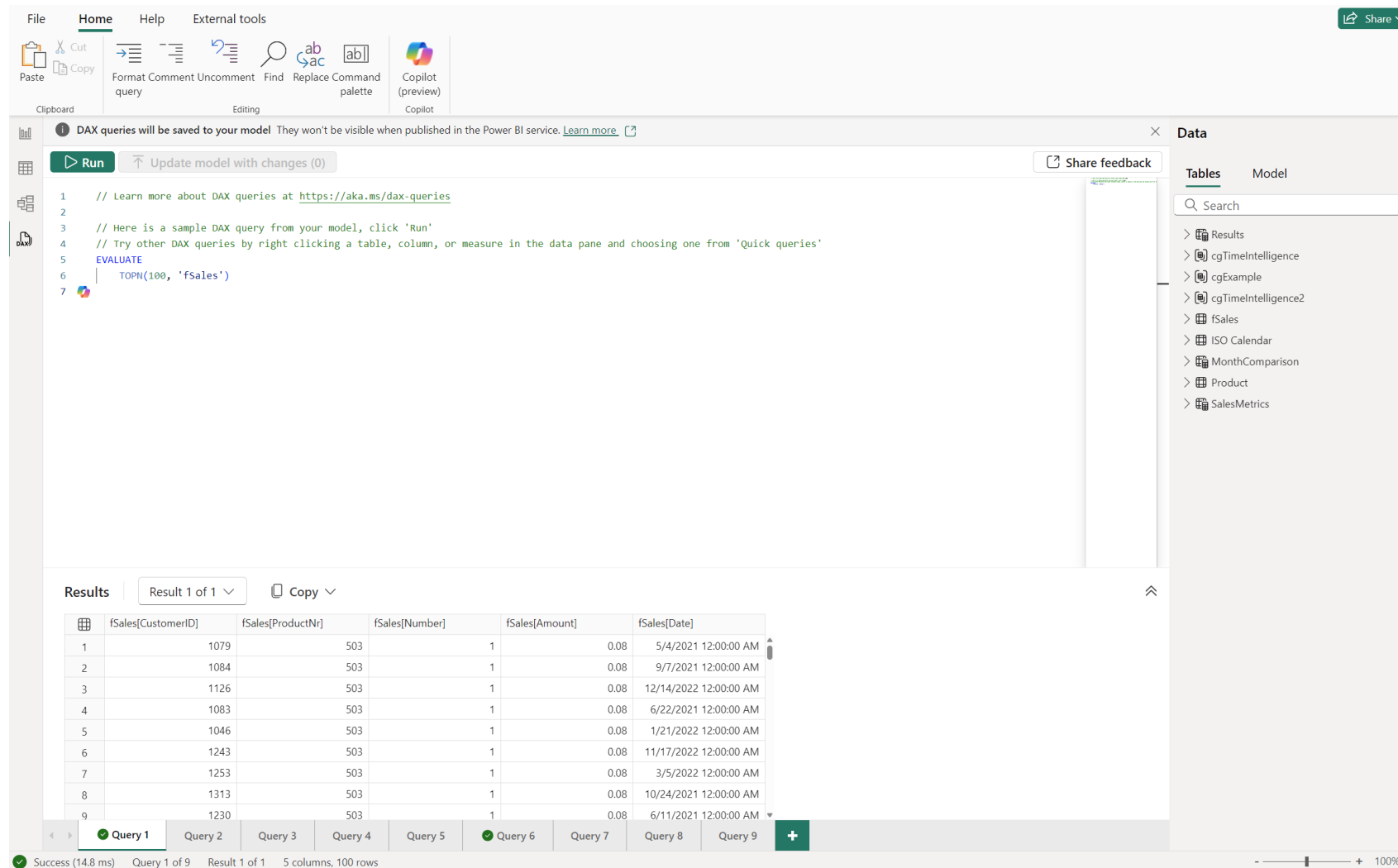
- A DAX Query always returns a table expression (unlike DAX measures!).
- Also returns the execution time of the query.



Possible uses

- Explore the model
- Modify or create DAX measures
- Modify or create DAX UDFs!
- Run queries from the Performance Analyzer
- Debug your model
- Identify slow measures

DAX Query View layout



DAX Query View layout

The screenshot shows the Power BI Desktop interface with the DAX Query View active. The ribbon at the top includes File, Home, Help, and External tools. The Home ribbon has several groups of icons: Clipboard (Paste, Cut, Copy), Editing (Format, Comment, Uncomment, Find, Replace, Command palette), and Copilot (Copilot, Copilot (preview)).

The main area displays a DAX query editor with the following text:

```
// Learn more about DAX queries at https://aka.ms/dax-queries
// Here is a sample DAX query from your model, click 'Run'
// Try other DAX queries by right clicking a table, column, or measure in the data pane and choosing one from 'Quick queries'
EVALUATE
    TOPN(100, 'fSales')
```

The right pane shows the Data pane with a search bar and a list of tables and measures:

- Results
- cgTimeIntelligence
- cgExample
- cgTimeIntelligence2
- fSales
- ISO Calendar
- MonthComparison
- Product
- SalesMetrics

The bottom pane shows the Results pane with a table of data:

	fSales[CustomerID]	fSales[ProductNr]	fSales[Number]	fSales[Amount]	fSales[Date]
1	1079	503	1	0.08	5/4/2021 12:00:00 AM
2	1084	503	1	0.08	9/7/2021 12:00:00 AM
3	1126	503	1	0.08	12/14/2022 12:00:00 AM
4	1083	503	1	0.08	6/22/2021 12:00:00 AM
5	1046	503	1	0.08	1/21/2022 12:00:00 AM
6	1243	503	1	0.08	11/17/2022 12:00:00 AM
7	1253	503	1	0.08	3/5/2022 12:00:00 AM
8	1313	503	1	0.08	10/24/2021 12:00:00 AM
9	1230	503	1	0.08	6/11/2021 12:00:00 AM

The bottom status bar shows: Success (14.8 ms) Query 1 of 9 Result 1 of 1 5 columns, 100 rows

DAX Query Editor

- IntelliSense
- View measure definitions

```
EVALUATE  
SUMMARIZECOLUMNS (  
    'ISO Calendar'[ISO Year],  
    "CurrentYearSales", Results[CurrentYearSales],  
    "PreviousYearSales", Results[PreviousYearSales]  
)
```

Measure CurrentYearSales
SUM(fSales[Amount])

DAX Query Results

Query results are limited to 1 million values.

Results | Result 1 of 1 | Copy

ⓘ Limited to 250.000 rows

	ISO Calendar[Date]	Product[ProductNr]	fSales[CustomerID]	[Sales]
1	11/25/2021 12:00:00 AM	308	1174	0.17
2	2/1/2022 12:00:00 AM	308	1196	0.08
3	6/14/2021 12:00:00 AM	308	1300	0.08
4	4/14/2022 12:00:00 AM	308	1189	0.08
5	8/24/2021 12:00:00 AM	308	1162	0.08
6	9/2/2021 12:00:00 AM	308	1196	0.08
7	6/18/2022 12:00:00 AM	308	1253	0.18
8	1/7/2021 12:00:00 AM	308	1148	0.15
9	6/26/2021 12:00:00 AM	308	1115	0.08

Query 1 Query 2 Query 3 Query 4 Query 5 **Query 6** Query 7 Query 8 Query 9 Query 10 Query 11 Query 12 Query 13 Query 14 +

Success (2538.0 ms) Query 6 of 22 Result 1 of 1 4 columns, 250.000 rows

DAX Query Results

Multiple EVALUATE statements are possible.

DAX queries will be saved to your model They won't be visible when published in the Power BI service. [Learn more](#)

Run **Update model with changes (0)**

```

1  EVALUATE
2  SUMMARIZECOLUMNS(
3      'ISO_Calendar'[Date],
4      "Sales", 'Results'[Sales]
5  )
6
7  EVALUATE
8  SUMMARIZECOLUMNS(
9      'ISO_Calendar'[ISO Year], 'ISO_Calendar'[Period],
10     "Sales", Results[Sales],
11     "Sales Last Year New", 'Results'[Sales Last Year New]
12 )

```

Results **Result 1 of 2** **Copy**

	ISO_Calendar[Date]	[Sales]
1	12/31/2020 12:00:00 AM	53963.31
2	1/1/2021 12:00:00 AM	53853.19
3	1/2/2021 12:00:00 AM	41285.4
4	1/3/2021 12:00:00 AM	56281.25
5	1/4/2021 12:00:00 AM	45041.49
6	1/5/2021 12:00:00 AM	51655.85
7	1/6/2021 12:00:00 AM	62037.27

TMDL view

- Allows code-oriented editing of a semantic model
- Prerequisites:
 - Enable .pbip save option
 - Including TMDL format

DAX user-defined functions

DAX User-defined functions

- Native DAX functions:

`SUMX(Table1, Table1[Column1] ^ 2)`

- With user-defined function, you can build your own:

`fnMYFUNCTION(Table1, [Measure1])`

- UDFs are created using DAX

Note: DAX UDFs have been in public preview since the Sep 2025 release and went GA in May 2026.

UDF definition syntax

```
fnMyFunction =  
    (  
        Parameter 1,  
        Parameter 2  
    ) =>  
    <DAX code using parameters>
```

Create a DAX UDF in DQV

DEFINE

/// This is my first DAX UDF

FUNCTION fnMyFirstUDF =

(pA, pB) => pA + pB

EVALUATE

{ fnMyFirstUDF(5, 10) }

Exercises

1. Create a function 'fnMultiply' that takes two parameters and returns the product of these.
2. Create a function 'fnRandom' that takes no parameter and returns a random number between 10 and 100.
3. Create a function 'fnKPI' that takes a numeric parameter and returns Sales, Number Sold, or Average Price, depending on the parameter value.

Parameter types

Parameter type

Parameter types (optional, you don't have to use them)

- **Scalar** values
- **Table** values
- **References** (column, table, measure, calendar, any)

Parameter type

```
fnMyFunction =  
(  
    Parameter 1: SCALAR,  
    Parameter 2: ANYREF  
) =>  
<DAX code using parameters>
```

fnMyFunction(5, [Sales])

Parameter subtype

Parameter subtypes (optional, only for scalar type)

- Variant
- Int64 (whole number)
- Decimal (currency)
- Double
- String (text)
- DateTime
- Boolean (true/false)
- Numeric

Parameter subtype

```
fnMyFunction =  
(  
    Parameter 1: SCALAR BOOLEAN,  
    Parameter 2: INT64  
) =>  
<DAX code using parameters>
```

fnMyFunction(TRUE(), 18)

Exercises

1. Create a new version of your `fnMultiply` function that only accepts integer values.
2. Use both functions in DAX Query View. What differences do you notice?

Parameter passing mode

Parameter passing mode

The function DAX code is evaluated in the **context** in which the function is called.

Evaluation of function parameters depends on the mode (or passing mode):

- **Val** – eager evaluation
 - Evaluation at function call, result is constant in the function body
- **Expr** – lazy evaluation
 - Evaluation in function body

Passing mode is optional. Defaults depend on parameter type.

‘Val’ – eager evaluation

```
fnTableFilter1 =  
  (pTable: val, pColumn: val) =>  
    CALCULATETABLE(  
      pTable,  
      pColumn > 10  
    )
```

fnTableFilter1(Table1, Table1[Value])

What is the result?

‘Val’ – eager evaluation (2)

fnTableFilter1(Table1, Table1[Value]) is equivalent to:

```
VAR pTable = Table1
VAR pColumn = Table1[Value] ⚠
RETURN
CALCULATETABLE(
    pTable,
    pColumn > 10 ⚠
)
```

‘Expr’ – lazy evaluation

```
fnTableFilter2 =  
  (pTable: val, pColumn: expr) =>  
    CALCULATETABLE(  
      pTable,  
      pColumn > 10  
    )
```

fnTableFilter2(Table1, Table1[Value]) ??

‘Expr’ – lazy evaluation (2)

fnTableFilter2(Table1, Table1[Value]) is equivalent to:

```
VAR pTable = Table1
RETURN
CALCULATETABLE(
    pTable,
    Table1[Value] > 10
)
```

‘Expr’ – lazy evaluation (3)

```
fnTableFilter3 =  
  (pTable: expr, pColumn: expr) =>  
    CALCULATETABLE(  
      pTable,  
      pColumn > 10  
    )
```

fnTableFilter3(Table1, Table1[Value]) ??

‘Expr’ – lazy evaluation (4)

fnTableFilter3(Table1, Table1[Value]) is equivalent to:

```
CALCULATETABLE(  
    Table1,  
    Table1[Value] > 10  
)
```

Exercise

1. Create a function that takes a measure reference as a parameter and returns the total result of the measure for all years.
2. Create a measure that calls the function for sales.
3. Create a simple report to show your function in action.

Naming convention

VAL passing mode:

pvValue

EXPR passing mode:

pxColumn

DAX UDFs: context

Context in UDFs

- Calling a UDF does NOT change context
 - (calling a measure does)
- Row context -> row context
- Filter context -> filter context (with the same set of filters)
- Visual context -> visual context

Helper functions

TABLEOF()

Returns a reference to the **table** of an item

```
fnTableFilter4 =
    (pxColumn: COLUMNREF) =>
        CALCULATETABLE(
            TABLEOF(pxColumn),
            pxColumn > 10
        )
```

fnTableFilter4(Table1[Value])

??

NAMEOF()

Returns a reference to the **name** of an item

```
fnColumnStatistics =
    (pxColumn: COLUMNREF) =>
        VAR RowCount = COUNTROWS(TABLEOF(pxColumn))
        VAR TableName = NAMEOF(pxColumn, PARENT)
        VAR Message = "Table ## contains ## rows"
        RETURN
        SUBSTITUTE(
            SUBSTITUTE(Message, "##", TableName, 1),
            "##", RowCount, 2
        )
```

fnColumnStatistics(Table1[Value]) ??

IS...

Returns whether a (parameter's) value is of a specific type

ISBOOLEAN/ ISLOGICAL
ISCURRENCY/ISDECIMAL
ISDOUBLE
ISINT64/ ISINTEGER
ISNUMERIC/ISNUMBER
ISSTRING/ISTEXT
ISDATETIME

Use these for input validation:

IF(ISINTEGER(pvValue), ...

(Note: IF does not work with tables!)

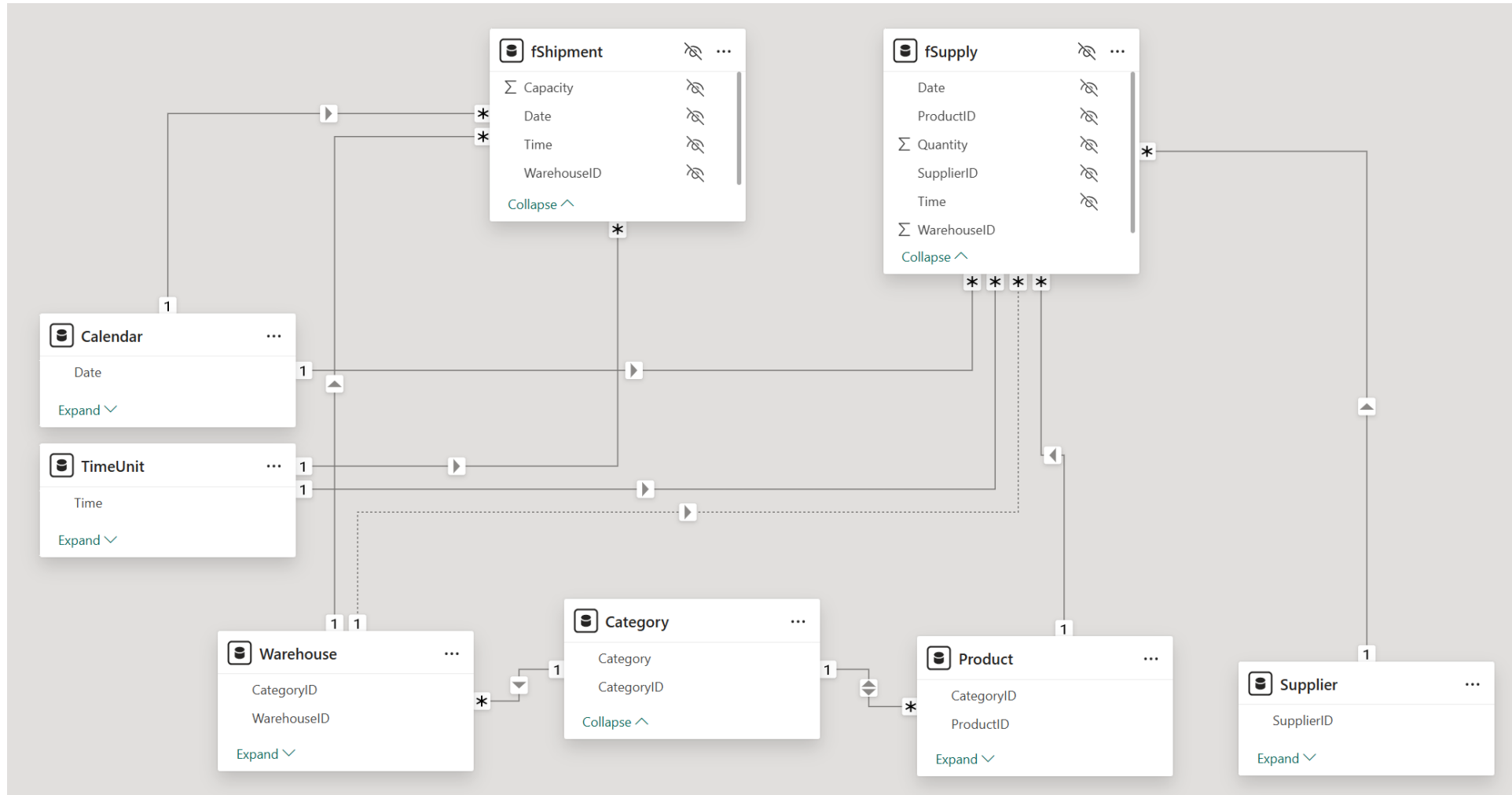
Exercise

1. Create a function that (only) takes a column reference as a parameter and returns the sum of squares of that column.
2. Create a function that takes a generic scalar value as a reference and that returns the square of the value when that value is numeric, or a random number between 1 and 1000 when it's not.

Things to remember

- A UDF can have a scalar value or a table as a result
- A UDF is evaluated in the context in which it is called
 - (row context, filter context, visual context)
- Parameters are either passed as value (VAL, pv, eager evaluation) or as expression (EXPR, px, lazy evaluation)
- For table calculations, TABLEOF eliminates the need for separate tableref parameters
- You may find useful UDFs at [Daxlib.org](https://daxlib.org)

DAX UDF example: Recursive calculations



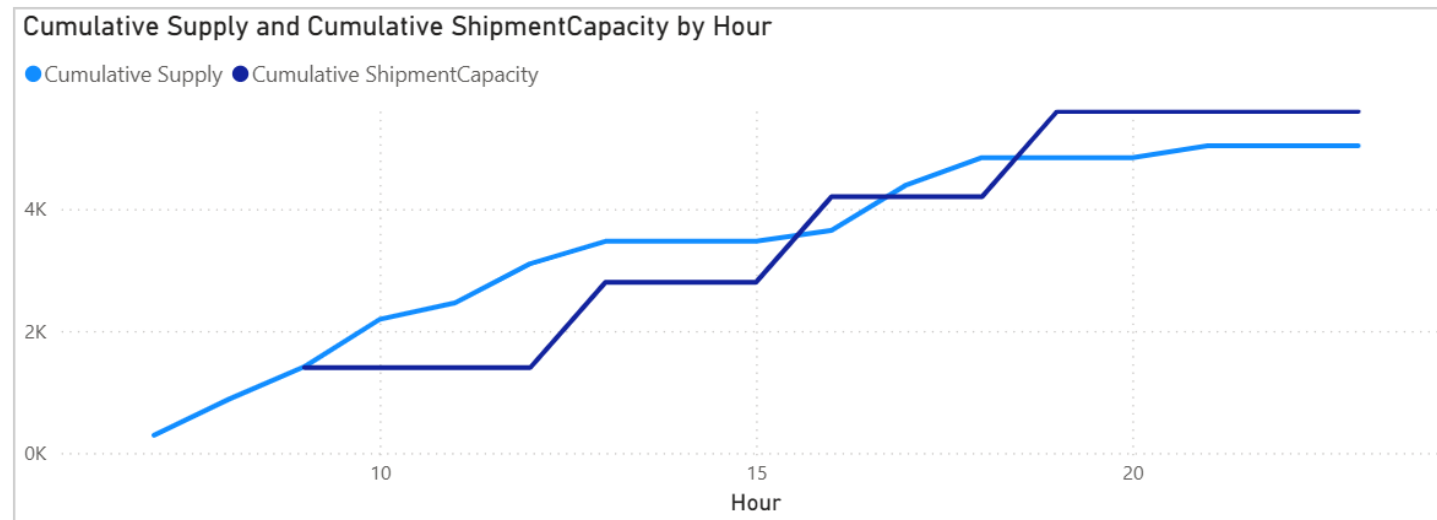
About the model

- Suppliers supply products to a warehouse
- Product are grouped in categories
- Warehouses are category-specific

- Shipments from warehouses have a maximum capacity
- Shipments are planned at specific points in time
- A shipment happens whether or not it's 100% full

Intra-day time intelligence

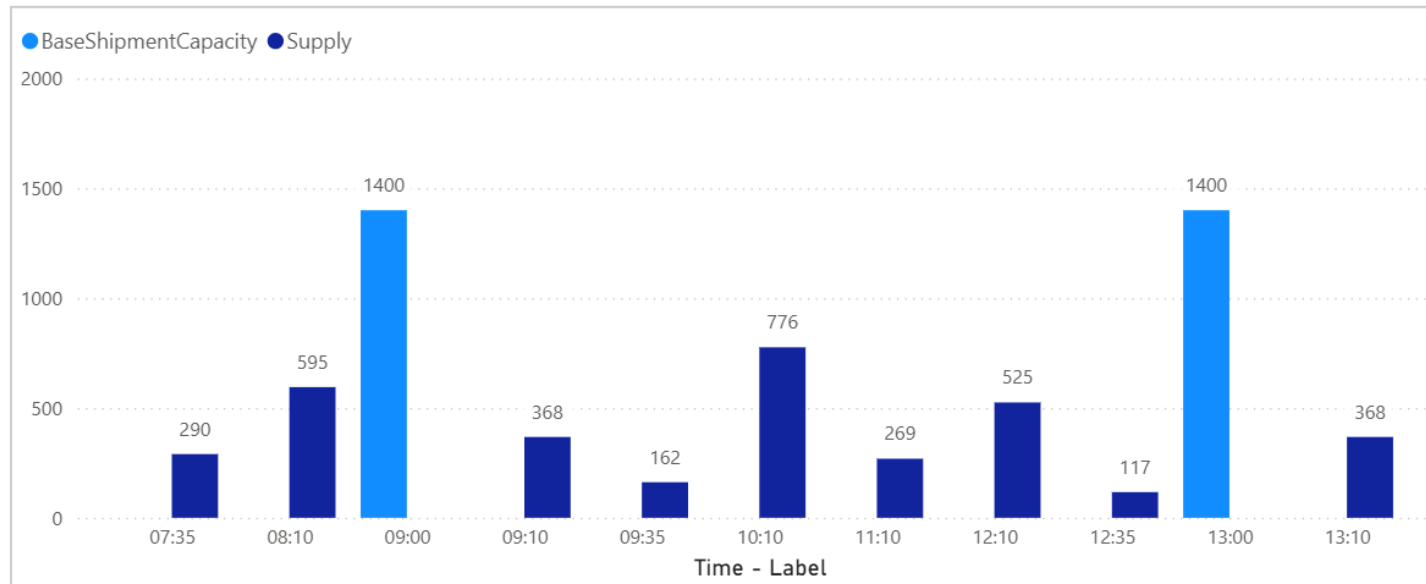
- We have created a custom calendar for time intelligence within a day
- This allows us to compute cumulative supplies and shipment capacity within a day



Question: what is the average shipment utilization?

Utilization depends on:

- Shipment capacity
- Available supplies at the time of shipment



We want to have a table with shipping metrics

CategoryID 	Hour 	Supply 	Storage before shipment 	Shipment capacity 	Shipped 	Shipped % 	Storage after shipment 
1	7		0		0	0%	0
1	8	290	290		0	0%	290
1	9	595	885	1400	885	100%	0
1	10	530	530		0	0%	530
1	11	776	1306		0	0%	1306
1	12	269	1575		0	0%	1575
1	13	642	2217	1400	1400	63%	817
1	14	368	1185		0	0%	1185
1	15		1185		0	0%	1185

Solve this with a UDF

Create a UDF `fnShippingMetrics` that computes the metrics for a single category and a single hour:

- All metrics returned as a single-row table
- Parameters for the hour and the initial storage (and perhaps others)

In the end, UNION all results to create the shipping metric table

We'll do this in DAX Query View!

DAX UDF example: Visual Calculations

What are visual calculations?

- DAX calculations executed in the scope of a single visual
- Easy and flexible: what you see is what you get
- Can sometimes help solve performance issues
- In support of visual calculations:
 - New concepts
 - New functions

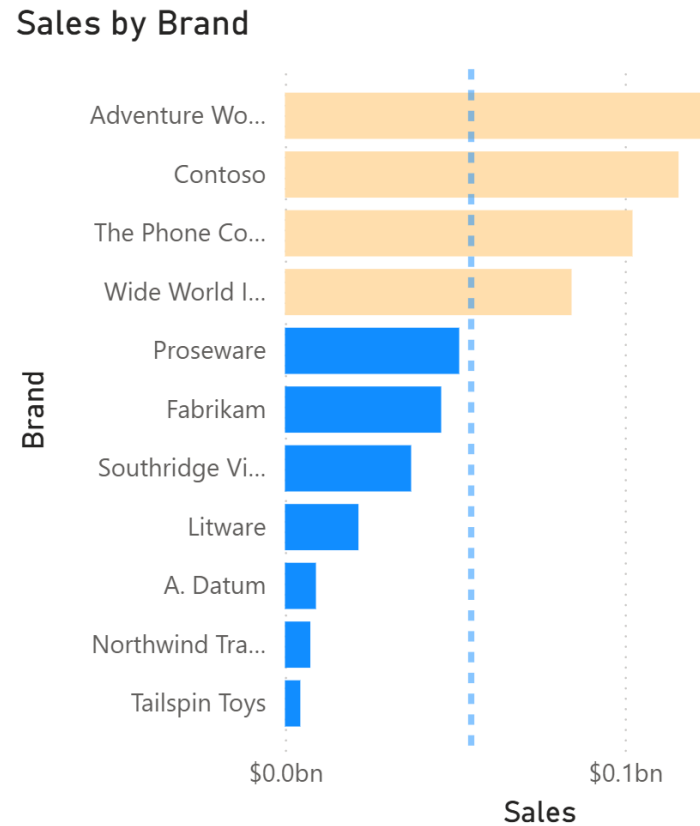
New concepts

- Visual calculations edit mode window
- Visual matrix
- Function templates
- Hiding intermediate results in a visual
- Axis
- Reset

Visual calculations

- Computed at the time of use
- Do not take up space in the model
- Use for
 - calculations that can easily be done with the exclusive functions
 - specific calculations for a single visual (like custom totals!)
 - conditional formatting of visuals
 - quick exploratory analytics/one-time calculations

Conditional formatting based on average



The biggest limitation

- Visual calculations are non-reusable 😞
- The solution: User-defined functions
- Approach:
 1. Put the logic of the visual calculation in a UDF
 2. Call the udf in a visual calculation in your visual

Example: coloring bars above average

Visual calculation:

ColorValue =

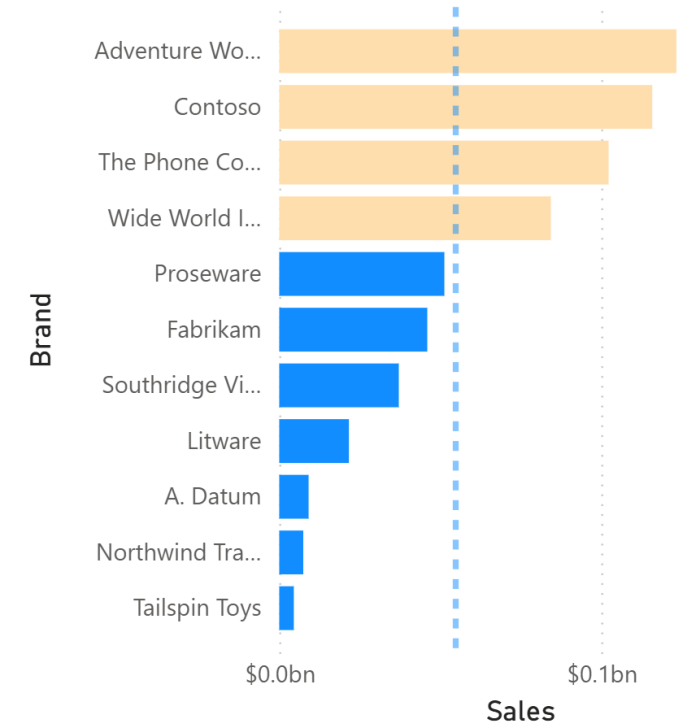
VAR AverageSales =

AVERAGEX(ROWS, [Sales])

RETURN

IF([Sales] > AverageSales, "Navajo White")

Sales by Brand

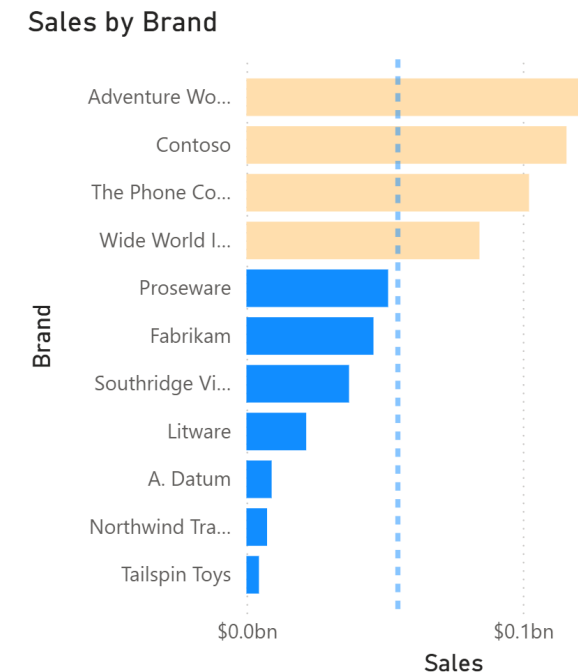


Example: coloring bars above average

User-defined function:

```
fnColorAboveAverage =
(
    pxColumn : COLUMNREF,
    pvColor : STRING
) =>
VAR AverageValue = AVERAGEX(ROWS, pxColumn)
RETURN
IF(pxColumn > AverageValue, pvColor)
```

ColorValue = fnColorAboveAverage([Sales], "Navajo White")



Note: data format must be set to 'text' for this to work

DAX UDF Example: DAX Queries

DAX Queries

- Used to provide other types of output than (visual) reports
- Outside process can call a semantic model with a DAX query
- Issue: many dependencies are created
- Solution: encapsulate DAX query logic as a UDF

EVALUATE fnOutput(*<optional filters here>*)

Thank you!